

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

06 - JavaScript: sintaxe moderna (ES6+)

Prof. Alex Kutzke

Setembro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Executar uma expressão no console do navegador e ler a resposta;
2. Ligar um arquivo .js externo com defer e confirmar que ele executou;
3. Escolher entre `const` e `let`, e dizer por que `var` ficou de fora;
4. Prever o resultado de uma comparação com `==` e com `===`;
5. Escrever uma função nas três formas de hoje e converter uma na outra;
6. Percorrer um array com `for...of` e com `forEach`;
7. Produzir arrays novos com `map` e `filter`, e um valor único com `reduce`;
8. Representar o catálogo como array de objetos e extrair um relatório;
9. Ler uma mensagem de erro do console e chegar à causa.

Roteiro

Console

Valores

Fluxo

Funções

Arrays e objetos

Erros

O console do navegador

F12, aba **Console**. O mesmo painel do inspetor de CSS.

```
2 + 3
```

```
"Loja" + " " + "Exemplo"
```

```
document.title
```

```
document.querySelectorAll("article.produto").length
```

Uma linha, um resultado, sem recarregar nada.

O que apareceu na última linha

- o código tinha acesso ao conteúdo da página, sem nada salvo em arquivo;
- `document` existia sem ter sido declarado;
- `querySelectorAll` e `length` vieram junto com ele.

Esse acesso à página chama-se DOM, e é o assunto do próximo encontro.

Hoje o console é a calculadora onde o código é conferido antes de ir para o arquivo.

Onde o JavaScript roda

Linguagem interpretada, de tipagem dinâmica, criada em 1995 para o navegador.

Cada navegador traz um motor: V8 (Chrome, Edge), SpiderMonkey (Firefox), JavaScriptCore (Safari).

Desde 2009 o V8 também roda fora do navegador, no Node.js. Neste semestre o servidor é PHP, e o JavaScript fica no cliente.

ECMAScript e o que significa ES6

A especificação se chama **ECMAScript**. JavaScript é a implementação que os navegadores entregam.

ES6, ou ES2015, reorganizou a linguagem: `let`, `const`, arrow functions, template strings, classes e promessas (*promises*).

Desde 2015 sai uma versão por ano, com acréscimos menores.

Tutorial anterior a 2015 usa `var` e outras construções já substituídas. Reconhecer o estilo antigo evita copiá-lo.

O script na página

<!-- 1. No atributo do elemento. Não use. -->

`<button onclick="alert('oi')">Clique</button>`

<!-- 2. Dentro da página. Só para teste rápido. -->

`<script>`

`console.log("rodou");`

`</script>`

<!-- 3. Arquivo externo. É o que usamos. -->

`<script src="js/app.js" defer></script>`

Estrutura no HTML, aparência no CSS, comportamento no JavaScript.

Por que defer

Sem atributo nenhum, o navegador para de montar a página, baixa o arquivo e executa. O script roda antes de a página existir.

defer: baixa em paralelo, executa depois que a página está montada.

```
<head>
  <link rel="stylesheet" href="css/estilo.css">
  <script src="js/produtos.js" defer></script>
  <script src="js/app.js" defer></script>
</head>
```

Vários scripts com defer executam na ordem em que aparecem.

Confirmar que o arquivo carregou

Caminho errado em `src` não aparece na tela: a página abre sem comportamento nenhum.

1. a primeira linha do arquivo é um `console.log`, e a mensagem aparece ao recarregar;
2. na aba **Network**, existe uma linha para o arquivo com status 200.

Status 404 ali significa caminho errado.

Prática: o primeiro arquivo

No seu *fork* da tarefa, no `catalogo.html`, seu ou da pasta `base/`:

1. crie `js/app.js` com um `console.log` na primeira linha;
2. ligue o arquivo no `<head>`, com `defer`;
3. recarregue e confira o console e a aba **Network**;
4. troque o `src` por `js/apps.js`, recarregue e anote o que mudou.

Tempo: 5 minutos.

Resposta: o primeiro arquivo

```
// js/app.js  
console.log("app.js carregou");
```

```
<link rel="stylesheet" href="css/estilo.css">  
<script src="js/app.js" defer></script>
```

Com js/apps.js, a mensagem some do console e a aba **Network** mostra 404 para o arquivo. A página abre igual.

Desfaça a troca antes de seguir.

Tipagem dinâmica

O tipo pertence ao valor, não à variável.

```
typeof "Café"      // "string"
typeof 3.14         // "number"
typeof false       // "boolean"
typeof undefined   // "undefined"
typeof [1, 2, 3]    // "object"
typeof null        // "object"
```

Array é um tipo de objeto. Para perguntar se um valor é array:
`Array.isArray(valor)`.

`typeof null` é defeito de 1995, nunca corrigido para não quebrar código existente.

undefined e null

undefined é o que o interpretador coloca. null é o que você coloca.

```
let preco;  
console.log(preco);           // undefined  
  
const produto = { nome: "Café" };  
console.log(produto.desconto); // undefined  
  
let selecionado = null;       // ninguém selecionado
```

Variável sem valor, propriedade que não existe e função sem return devolvem undefined.

const e let

const por padrão. let quando o valor for reatribuído.

```
const taxa = 0.1;
let total = 0;

total = total + 100;    // válido
taxa = 0.2;             // TypeError
```

Escrever `preco = 10` sem declarar cria uma global silenciosa.

const com array e objeto

const protege a ligação entre o nome e o valor, não o conteúdo.

```
const carrinho = ["café", "chá"];
```

```
carrinho.push("mate");    // válido  
carrinho = ["outro"];     // TypeError
```

Array declarado com const continua aceitando push, pop e atribuição por índice.

Escopo de bloco

Bloco é qualquer trecho entre chaves.

```
let x = "fora";

if (true) {
  let x = "dentro";
  console.log(x);    // dentro
}

console.log(x);      // fora
```

Por que var ficou de fora

```
function testa() {  
  for (var i = 0; i < 3; i++) {  
    // ...  
  }  
  console.log(i);  // 3, o i sobreviveu ao for  
}
```

- var tem escopo de função, e vaza para fora do bloco;
- sofre *hoisting*: lido antes da declaração, devolve undefined em vez de erro.

Aparece aqui para ser reconhecido em código alheio.

Template literals

Crase, com `${}` em volta de cada expressão.

```
const nome = "Café em grão";
const preco = 32.5;

// concatenação, o jeito antigo
console.log("0 " + nome + " custa R$ " + preco + ".");

// template literal
console.log(`0 ${nome} custa R$ ${preco}.`);

// qualquer expressão serve
console.log(`Com 10% off: R$ ${preco * 0.9}.`);
```

Strings

```
const termo = "  Café Em Grão  ";

termo.trim()                // "Café Em Grão"
termo.trim().toLowerCase()  // "café em grão"
"café em grão".includes("grão") // true
"café em grão".startsWith("café") // true
```

Strings são imutáveis: todo método devolve uma string nova, e o resultado precisa ser guardado ou usado ali mesmo.

Busca que ignora maiúsculas e minúsculas:

```
nome.toLowerCase().includes(busca.toLowerCase())
```

Números são ponto flutuante

```
0.1 + 0.2           // 0.30000000000000004
```

O mesmo acontece em C, Java e Python: 0,1 não tem representação exata em base 2. Em valores monetários, arredonde na hora de mostrar ou guarde sempre em centavos.

```
const total = 32.5 * 3;           // 97.5

total.toFixed(2)                   // "97.50", string
Number(total.toFixed(2))           // 97.5

(1890).toLocaleString("pt-BR",
  { style: "currency", currency: "BRL" });
// "R$ 1.890,00"
```

Texto que vira número

Campo de formulário devolve sempre string.

```
"10" + 5           // "105", concatenação
```

```
"10" - 5           // 5, o - converte
```

```
Number("10")       // 10
```

```
Number("10,5")      // NaN
```

```
Number("")          // 0
```

```
parseFloat("10.5 reais") // 10.5
```

```
parseInt("42px")     // 42
```

NaN é do tipo number e não é igual a si mesmo.

Teste com `Number.isNaN(valor)`.

=== e ==

=== compara valor e tipo. == converte antes de comparar.

```
1 == "1"           // true
1 === "1"          // false

0 == false         // true
0 === false        // false

null == undefined  // true
null === undefined // false
```

Use === e !== sempre.

Os seis valores falsos

```
false  
0  
""           // string vazia  
null  
undefined  
NaN
```

Todo o resto vira true, inclusive "0", "false", [] e {}.

Array vazio dentro de um if entra no bloco.

Decisão

```
if (preco > 100) {  
  console.log("caro");  
} else if (preco > 20) {  
  console.log("médio");  
} else {  
  console.log("barato");  
}
```

```
const frete = preco > 100 ? 0 : 15;
```

```
if (preco >= 20 && preco <= 50) { /* ... */ }
```

Operadores lógicos: &&, || e !.

Repetição

```
const frutas = ["café", "chá", "mate"];

// quando o índice importa
for (let i = 0; i < frutas.length; i++) {
  console.log(i, frutas[i]);
}

// quando só o valor importa
for (const fruta of frutas) {
  console.log(fruta);
}
```

O contador é `let` porque é reatribuído. A variável do `for...of` recebe valor novo a cada volta, e pode ser `const`.

Três formas de escrever

// declaração

```
function calculaTotal(preco, quantidade) {  
  return preco * quantidade;  
}
```

// expressão de função

```
const calculaTotal = function (preco, quantidade) {  
  return preco * quantidade;  
};
```

// arrow function (ES6)

```
const calculaTotal = (preco, quantidade) => {  
  return preco * quantidade;  
};
```

Arrow function, formas curtas

```
// corpo de uma expressão: chaves e return somem juntos  
const calculaTotal = (preco, qtd) => preco * qtd;
```

```
// um parâmetro: parênteses opcionais  
const dobro = valor => valor * 2;
```

```
// nenhum parâmetro: parênteses obrigatórios  
const agora = () => new Date();
```

A armadilha do retorno implícito

```
const dobro = valor => { valor * 2 };
```

```
dobro(4);    // undefined
```

Com chaves, o return precisa estar escrito. Sem chaves, o valor da expressão é devolvido.

É um dos defeitos mais frequentes, e ele não gera erro nenhum no console.

Parâmetro com valor padrão

```
const calculaTotal = (preco, quantidade = 1) =>  
  preco * quantidade;
```

```
calculaTotal(32.5);      // 32.5  
calculaTotal(32.5, 3);  // 97.5
```

Chamar com menos argumentos do que os parâmetros não é erro: os que faltam ficam undefined, e a conta devolveria NaN.

Função é um valor

```
const aplicar = (valor, operacao) => operacao(valor);
```

```
aplicar(4, dobro);           // 8  
aplicar(4, valor => valor + 1); // 5
```

Função guardada em variável, passada como argumento, devolvida por outra função.

Função passada como argumento chama-se *callback*, e é o que torna possível o `forEach` da próxima seção.

Array

```
const precos = [32.5, 18, 74.9, 12];
```

```
precos[0]           // 32.5
precos.length       // 4
precos[precos.length - 1] // 12
precos[10]          // undefined
```

```
precos.push(50);    // acrescenta no fim
precos.pop();       // remove do fim
precos.unshift(5);  // acrescenta no começo
precos.shift();     // remove do começo
```

Objeto

```
const produto = {  
  id: 1,  
  nome: "Café em grão",  
  categoria: "Bebidas",  
  preco: 32.5,  
  desconto: 0.15  
};  
  
produto.nome           // "Café em grão"  
produto["nome"]       // igual  
  
const campo = "preco";  
produto[campo]        // 32.5
```

Colchete quando o nome da chave vem de uma variável.

Array de objetos

É o formato em que os dados chegam de uma API, e o do seu catálogo.

```
const produtos = [
  { id: 1, nome: "Café em grão",   preco: 32.5 },
  { id: 2, nome: "Chá de hibisco",  preco: 18   },
  { id: 3, nome: "Mel silvestre",   preco: 24.9 }
];
```

```
produtos.length           // 3
produtos[0].nome          // "Café em grão"
```

```
for (const produto of produtos) {
  console.log(produto.nome);
}
```

Prática: os produtos como dados

1. crie `js/produtos.js` com um array `produtos`, um objeto por produto do catálogo, com `id`, `nome`, `categoria`, `preco` e `desconto`;
2. ligue esse arquivo **antes** do `app.js`;
3. no `app.js`, imprima quantos produtos existem.

O número tem que bater com o de cartões na página. Na pasta `base/`, são seis; o desconto de cada um fica a seu critério.

Tempo: 8 minutos.

Resposta: os produtos como dados

```
const produtos = [  
  { id: 1, nome: "Café em grão", categoria: "Bebidas", preco: 32.5, desconto: 0.15 },  
  { id: 2, nome: "Chá de hibisco", categoria: "Bebidas", preco: 18, desconto: 0 },  
  { id: 3, nome: "Mel silvestre", categoria: "Mercearia", preco: 24.9, desconto: 0.1 },  
  { id: 4, nome: "Farinha de milho", categoria: "Mercearia", preco: 12, desconto: 0 },  
  { id: 5, nome: "Sabonete de argila", categoria: "Higiene", preco: 9.9, desconto: 0.2 },  
  { id: 6, nome: "Cesta de vime", categoria: "Utensílios", preco: 74.9, desconto: 0 }  
];
```

```
// app.js  
console.log(produtos.length);    // 6  
console.table(produtos);
```

`console.table` monta uma linha por item e uma coluna por campo. É a forma mais rápida de conferir um array de objetos.

Percorrer: forEach

```
const produtos = ["Café", "Chá", "Mate"];

produtos.forEach(produto => console.log(produto));

produtos.forEach((produto, indice) => {
  console.log(`${indice}: ${produto}`);
});
```

Não devolve nada e não pode ser interrompido: `break` não funciona dentro dele. Para parar antes do fim, `for...of`.

Prática: uma linha por produto

Com `forEach`, imprima no console uma linha por produto, neste formato:

Café em grão: R\$ 32.50

Chá de hibisco: R\$ 18.00

Tempo: 5 minutos.

Resposta: uma linha por produto

```
produtos.forEach(produto => {  
  console.log(`${produto.nome}: R$ ${produto.preco.toFixed(2)}`);  
});
```

toFixed(2) garante as duas casas: sem ele, o chá sairia como R\$ 18.

Transformar: map

Devolve um array **novo**, do mesmo tamanho.

```
const precos = [32.5, 18, 74.9];

const comDesconto = precos.map(preco => preco * 0.9);
// [29.25, 16.2, 67.41000000000001]

precos; // [32.5, 18, 74.9], inalterado
```

A cauda de casas do terceiro valor é o ponto flutuante da seção anterior. Ela some na exibição, com `toFixed` ou `toLocaleString`.

Prática: preço com desconto

Com `map`, produza o array `precosFinais`: o preço de cada produto com o desconto já aplicado.

O array `produtos` não pode mudar. Confira depois com `console.table(produtos)`.

Tempo: 5 minutos.

Resposta: preço com desconto

```
const precosFinais = produtos.map(  
  produto => produto.preco * (1 - produto.desconto)  
);  
// [27.625, 18, 22.41, 12, 7.920000000000001, 74.9]
```

Confira o primeiro na calculadora: R\$ 32,50 com 15% de desconto.

Produto com desconto: 0 sai com o mesmo preço, e está correto.

Selecionar: filter

Devolve um array novo com os elementos que passaram no teste.

```
const precos = [32.5, 18, 74.9, 12];  
  
const baratos = precos.filter(preco => preco < 30);  
// [18, 12]
```

A função precisa devolver true ou false. Escrever a condição entre chaves e esquecer o return devolve array vazio, porque undefined é valor falso.

Prática: busca e filtro

Com `filter`, escreva duas consultas:

1. produtos cujo nome contém um termo, sem diferenciar maiúsculas de minúsculas. Teste com "ES";
2. produtos com preço entre 20 e 50, incluindo os extremos.

São dois requisitos da Entrega 2 do trabalho prático.

Tempo: 8 minutos.

Resposta: busca e filtro

```
const termo = "ES";
const encontrados = produtos.filter(
  produto =>
    ↪ produto.nome.toLowerCase().includes(termo.toLowerCase())
);
// Mel silvestre, Cesta de vime

const naFaixa = produtos.filter(
  produto => produto.preco >= 20 && produto.preco <= 50
);
// Café em grão, Mel silvestre
```

Resumir: reduce

```
const precos = [32.5, 18, 74.9, 12];

const soma = precos.reduce(
  (acumulado, preco) => acumulado + preco,
  0
);
// 137.4
```

acumulado é o que voltou da chamada anterior; na primeira volta, é o segundo argumento.

Escreva sempre o valor inicial.

Prática: valor total

Com `reduce`, some o preço de todos os produtos, sem desconto.

Tempo: 4 minutos.

Resposta: valor total

```
const total = produtos.reduce(  
  (soma, produto) => soma + produto.preco,  
  0  
);  
// 172.20000000000002  
  
total.toFixed(2)   // "172.20"
```

O acumulador começa em 0 e recebe um preço a cada volta.

Prática: produtos por categoria

Monte um objeto com a quantidade de produtos de cada categoria, percorrendo o array com `for...of`:

```
// resultado esperado com as páginas de base/  
{ Bebidas: 2, Mercearia: 2, Higiene: 1, Utensílios: 1 }
```

Comece com `const contagem = {};` e use `contagem[produto.categoria]`.

Tempo: 8 minutos.

Resposta: produtos por categoria

```
const contagem = {};  
  
for (const produto of produtos) {  
  if (contagem[produto.categoria] === undefined) {  
    contagem[produto.categoria] = 0;  
  }  
  contagem[produto.categoria] = contagem[produto.categoria] + 1;  
}
```

Colchete porque o nome da chave vem da variável. Na primeira vez que uma categoria aparece, a chave ainda não existe e o valor lido é `undefined`.

A mesma contagem feita com `reduce` está na seção 20 do texto da aula.

Procurar

```
const precos = [32.5, 18, 74.9];

precos.includes(18);           // true
precos.find(preco => preco > 30); // 32.5
precos.findIndex(preco => preco > 30); // 0
precos.some(preco => preco > 70);  // true
precos.every(preco => preco > 10); // true
```

`find` devolve o elemento, ou `undefined`. `filter` devolve sempre um array.

Encadear

```
const total = precos
  .filter(preco => preco < 50)    // [32.5, 18, 12]
  .map(preco => preco * 0.9)      // [29.25, 16.2, 10.8]
  .reduce((soma, p) => soma + p, 0);
// 56.25
```

map e filter devolvem arrays, então as chamadas se encadeiam e a leitura é da esquerda para a direita.

Ordenar: sort

sort altera o array original e, sem argumento, ordena como texto.

```
const precos = [32.5, 18, 74.9, 120];

precos.sort(); // [120, 18, 32.5, 74.9]
precos.sort((a, b) => a - b); // [18, 32.5, 74.9, 120]
precos.sort((a, b) => b - a); // decrescente
```

A comparação devolve negativo quando a vem antes, positivo quando b vem antes, zero quando tanto faz.

Desestruturação

```
const { nome, preco } = produto;  
  
const linha = ({ nome, preco }) =>  
  `${nome}: R$ ${preco.toFixed(2)}`;
```

As chaves do lado esquerdo criam uma constante para cada campo com o mesmo nome. Na lista de parâmetros, fazem o mesmo com o objeto recebido.

A mensagem no console

Diferente do CSS, que ignora em silêncio, o JavaScript interrompe e escreve.

```
Uncaught TypeError: produtos.filtrar is not a function  
    at app.js:14:20
```

SyntaxError	chave ou aspas sem fechar; nada roda
ReferenceError	nome não declarado, ou digitado errado
TypeError	o valor existe, mas não é do tipo pedido

O erro mais comum

```
const produto = produtos.find(p => p.id === 99);  
console.log(produto.nome);  
// TypeError: Cannot read properties of undefined  
//   (reading 'nome')
```

O find não encontrou e devolveu undefined. O erro só apareceu na linha seguinte, quando alguém pediu .nome daquilo.

A causa costuma estar uma linha acima da linha apontada.

Quando não há erro nenhum

Resultado errado, console limpo. Três conferências:

1. `console.log` do valor antes e depois da operação suspeita;
2. `typeof valor`, para descobrir que o número era uma string;
3. painel **Sources**, ponto de parada na linha, página recarregada: a execução para ali e o painel mostra cada variável.

O terceiro item é o depurador, e ele volta com mais uso quando entrarmos em eventos.

Prática: ler o erro

Quebre o `app.js` de propósito, uma quebra por vez. Recarregue, anote o tipo do erro e a linha, e desfaça:

1. troque `produtos.filter` por `produtos.filtrar`;
2. apague uma chave `}` no meio do arquivo;
3. busque com `find` o `id 999` e imprima o `.nome` do resultado;
4. escreva `const dobro = (v) => { v * 2 }`; e imprima `dobro(4)`.

Tempo: 8 minutos.

Resposta: ler o erro

-
- 1 `TypeError: produtos.filtrar is not a function`
 - 2 `SyntaxError: nada do arquivo roda`
 - 3 `TypeError: Cannot read properties of undefined`
 - 4 `nenhum erro: imprime undefined`
-

No terceiro, a causa está na linha do `find`, uma acima da apontada.

O quarto é o retorno implícito com chaves. O console não denuncia, e o defeito só aparece quando alguém confere o valor.

Tarefa

Os exercícios valem nota e compõem o item *Exercícios em sala*.

Enunciado no README.md do repositório-modelo; endereço e prazo na UFPR Virtual.

Entrega pelo *fork* no GitLab, individual ou em dupla, sem uso de IA generativa para produzir o código.

A busca e o filtro de hoje são dois requisitos da Entrega 2 do trabalho prático, que vence na aula anterior à Prova 2.